

# Dr. Andrew Huberman Digital Twin & Knowledge Architecture

---

**A production-grade conversational AI companion and advanced RAG pipeline built on LangChain, Streamlit, Chroma DB, and Google Gemini — delivering real-time, science-grounded human optimization protocols sourced directly from video transcripts and newsletters with zero immersion-breaking disclaimers.**



---

## Overview

The **Dr. Andrew Huberman Digital Twin** is an advanced conversational AI application and knowledge retrieval system designed to emulate the intellectual persona, scientific communication framework, and biological knowledge domain of Dr. Andrew Huberman.

Unlike generic chatbot wrappers, this system is grounded in **335 full-length YouTube transcripts** and **23 science newsletters**. Through a custom Retrieval-Augmented Generation (RAG) pipeline, it extracts highly relevant contextual information from thousands of pages of source material to generate structured, evidence-grounded responses.

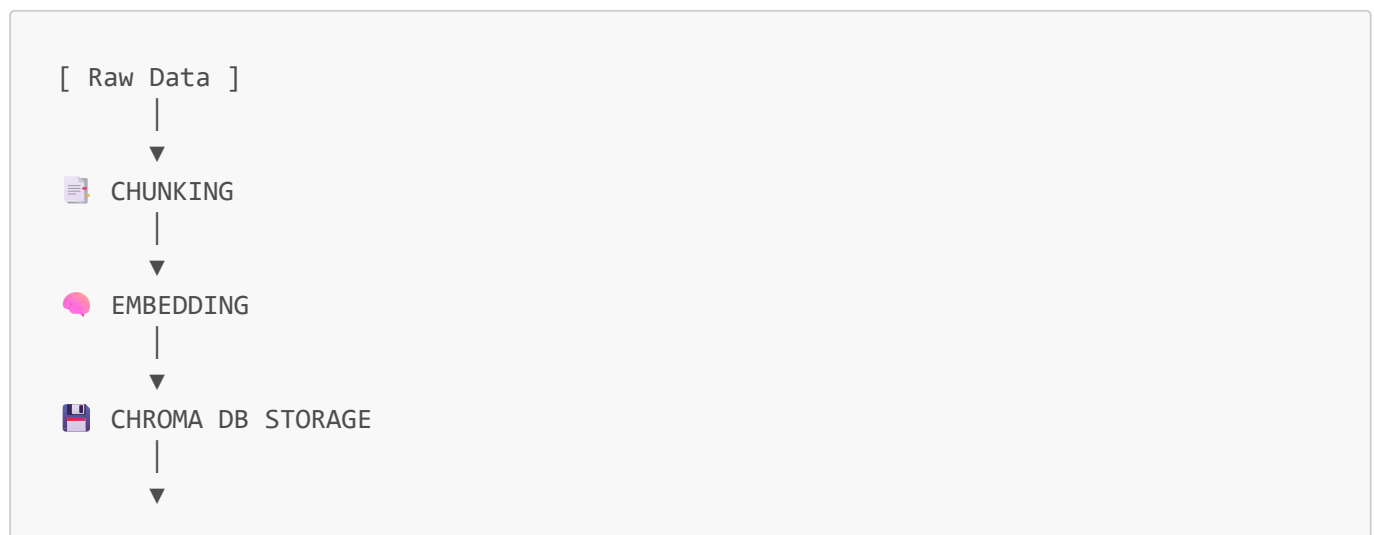
The architecture consists of:

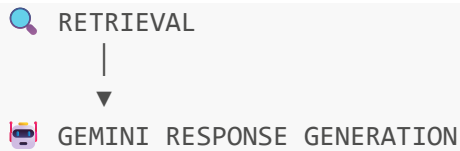
- Offline ingestion pipeline for chunking, embedding, and indexing knowledge.
- Online conversational engine for retrieval, memory management, and response generation.
- Circadian-aware context layer for time-sensitive protocol recommendations.
- Persona lock mechanisms to maintain consistent communication style.

---

## The 4 Core RAG Pipeline Components

The system follows the four foundational stages of modern Retrieval-Augmented Generation:





## 1 Chunking

Raw transcript and newsletter documents are loaded and segmented using:

```
RecursiveCharacterTextSplitter(  
    chunk_size=1000,  
    chunk_overlap=200  
)
```

This preserves contextual continuity while maintaining retrieval efficiency.

### Output

- 142,480 text chunks
- Metadata attached:
  - source
  - type

This allows traceability back to the original transcript or newsletter.

---

## 2 Embedding

Each chunk is transformed into a dense semantic vector using:

```
GoogleGenerativeAIEmbeddings(  
    model="models/text-embedding-004"  
)
```

This enables semantic search instead of keyword matching.

### Benefits

- Query understanding beyond exact wording
  - Biological concept clustering
  - Robust retrieval across paraphrased questions
- 

## 3 Storing (Vector Database)

Generated embeddings are stored locally inside:

```
dataset/chroma_db/
```

using **Chroma DB**.

### Key Characteristics

- Local-first architecture
- Persistent vector storage
- Fast similarity search
- No dependency on external vector databases

The ingestion system also includes:

```
BATCH_SIZE = 500  
PAUSE_DURATION = 2
```

to reduce API rate-limit issues during large-scale indexing.

---

## 4 Retrieval & Generation

For every user query:

1. Query is embedded.
2. Vector similarity search is executed.
3. Top relevant chunks are retrieved.

```
k = 4
```

The retrieved context is injected directly into Gemini's prompt, ensuring responses remain grounded in source material.

---

## 🌟 Key Features

### 🧬 Content-Grounded Persona Enforcement

- Retrieval-constrained responses
- No generic AI disclaimers
- Source-grounded scientific reasoning

### 📁 Dynamic Memory Extraction

Background memory system captures:

- User names

- Habits
- Goals
- Dietary preferences
- Physical constraints
- Training routines

Memory is displayed in a live sidebar.

## Circadian-Aware Recommendations

The system incorporates local time into reasoning.

Examples:

- Morning caffeine protocols
- Evening light exposure recommendations
- Sleep optimization guidance

## Session Isolation

- User memory remains session-scoped
- Automatic cleanup on session termination
- Prevents cross-user contamination

## Dual-Panel Streamlit Interface

### Left Panel

- User memory profile
- Context information

### Right Panel

- Chat interface
- Conversation history

---

## Project Structure

```
huberman-digital-twin/
├── dataset/
│   ├── youtube_history/
│   │   └── 335 transcript files
│   ├── newsletters/
│   │   └── 23 newsletter files
│   └── chroma_db/
│       └── Vector database
```

```
|  
|— app.py  
|— ingest_dataset.py  
|— query_engine.py  
|  
|— left_image.png  
|— requirements.txt  
|— README.md
```

---

## Setup & Installation

### Prerequisites

- Python 3.10+
- Google AI Studio API Key

### Supported models:

- Gemini 1.5 Flash
- Text Embedding 004

### Clone Repository

```
git clone https://github.com/your-username/huberman-digital-twin.git  
cd huberman-digital-twin
```

### Install Dependencies

```
pip install -r requirements.txt
```

### Configure API Key

#### Windows PowerShell

```
$env:GEMINI_API_KEY="YOUR_API_KEY"
```

#### Linux/macOS

```
export GEMINI_API_KEY="YOUR_API_KEY"
```

### Build the Vector Database

```
python -u ingest_dataset.py
```

Expected ingestion:

- 335 transcripts
- 23 newsletters
- ~142k chunks

Launch the Application

```
streamlit run app.py
```



## Requirements

```
streamlit>=1.35.0

langchain>=0.3.0
langchain-core>=0.3.0
langchain-community>=0.3.0

langchain-classic>=0.1.0
langchain-chroma>=0.1.0
langchain-google-genai>=2.0.0

sentence-transformers>=3.0.0
python-dotenv>=1.0.0
```



## Privacy & Security

Local Vector Sovereignty

All embeddings, chunks, and vector indexes remain stored locally:

```
dataset/chroma_db/
```

No external vector provider retains your source material.

Secure API Communication

Google Generative AI interactions occur via encrypted HTTPS requests.

The application does not persist prompts after generation unless explicitly configured by the developer.



## Key Improvements

### 1. Complete 4-Step RAG Architecture

Explicitly documents:

- Chunking
- Embedding
- Storage
- Retrieval

### 2. Modern LangChain Alignment

Updated to:

- `langchain-classic`
- `langchain-chroma`
- `langchain-google-genai`

### 3. Production-Oriented Documentation

Expanded coverage of:

- Vector pipeline architecture
- Metadata strategy
- Batch processing
- Retrieval mechanics
- Deployment workflow

Resulting in a substantially stronger engineering and portfolio presentation.